Q-VGM: Q-Value-Gradient Matching for Off-Policy Reinforcement Learning of Flow-Matching VLA

Ziqian Wang[1,2] **Yitian Liu**[1] **Xingjian Mao**[1] **Minqian Wang**[1,2] **Yao Mu**[1†]

[1]Shanghai Jiao Tong University
[2]University of Michigan, Ann Arbor
[†]Corresponding author.

Abstract

We propose *Q-Guided Value-Gradient Matching* (Q-VGM), an off-policy reinforcement learning method for a central difficulty in fine-tuning flow-matching vision-language-action (VLA) policies: improving an expressive flow-matching action expert with a learned Q-function. Effective improvement must exploit the critic's first-order signal $\nabla_A Q$, yet flow policies make this hard: backpropagating values through the multi-step denoising chain is unstable at VLA scale, and the tractable action likelihoods required by policy gradients are unavailable under iterative denoising. Existing value-based methods therefore backpropagate through the full chain, use the critic only for test-time selection or guidance, or distill critic-improved actions as terminal labels that never supervise the velocity field. Q-VGM instead casts policy improvement as optimal control over the denoising dynamics, where the optimal residual velocity is the gradient of a denoising-time value function: clean-action estimates improved by iterative Q-gradient ascent with keep-best selection are converted into residual velocity targets that directly supervise the velocity field—no action likelihoods, no backpropagation through the denoising chain, and no critic at inference time. The critic is an action-sensitive stepwise IQL critic on compact latent states from the frozen VLA backbone. This enables a few-shot-initialization, learn-from-experience paradigm: starting from a few-shot-SFT $\pi_{0.5}$ policy, Q-VGM improves the policy from its own rollouts without additional expert supervision, raising the average LIBERO success rate from 79.0% to 92.5%, outperforming all same-backbone, same-critic baselines, and attaining high success rates on four real-robot manipulation tasks, including fine-grained plug insertion.

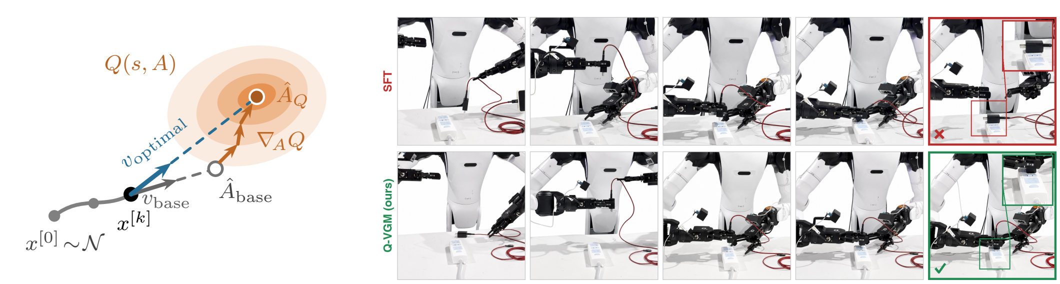

Figure 1: **Q-VGM turns critic gradients into velocity supervision.** Left: during offline fine-tuning, the action chunk predicted by the SFT velocity field (gray) is improved by ascending the critic gradient $\nabla_A Q$, and the induced displacement supervises the velocity field as a residual velocity target; no critic is used at deployment. Right: on a real-robot bimanual plug-insertion task, the SFT policy (top) misaligns the plug at the socket and fails; the Q-VGM fine-tuned policy (bottom) corrects the alignment and completes the insertion on the same task setup.

1 Introduction

Vision-language-action (VLA) models bridge high-level multimodal reasoning and low-level continuous control [1–5]. Flow-matching action experts have become their dominant backbone: by transporting noise to action chunks through iterative denoising, they capture the multimodal action distributions required for dexterous, long-horizon manipulation [4–7].

The prevailing approach of post-training for VLA models is supervised fine-tuning (SFT) on expert demonstrations, which inherits a fundamental limitation of imitation learning: it exploits only near-optimal expert data. A policy produces abundant failed and suboptimal rollouts during evaluation and deployment, yet imitation objectives provide no mechanism for learning from them. Consequently, policy performance is bounded by the quality and coverage of the demonstrations. Reinforcement learning (RL) provides a principled alternative. By directly optimizing task success, RL extracts learning signal from both successful and failed experience, allowing a policy to improve from its own rollouts and to exceed its demonstrator.

Realizing this potential for flow-matching VLAs, however, is constrained in two respects. The first concerns the data regime. On-policy methods such as PPO and GRPO require fresh rollouts from the current policy at every update, which is feasible in simulation but prohibitively expensive on real robots. They also discard previously collected experience after each update. The second concerns the parameterization. Policy-gradient methods require a tractable action likelihood, which iterative denoising does not expose. Recent methods therefore recover only approximate likelihoods through stochastic-flow variants or surrogate objectives [8–12]. Furthermore, policy gradients exploit only zeroth-order reward information (a scalar advantage), whereas the first-order signal $\nabla_A Q$ from a learned critic yields substantially better sample efficiency [13]. These considerations motivate off-policy, value-based RL, in which a critic trained on replay data improves the flow policy without requiring action likelihoods or on-policy interaction.

Existing value-based methods, however, fail to provide value-aware supervision for the flow policy itself. Backpropagating Q-gradients through the full denoising chain [14] is unstable and costly at VLA scale. Test-time Q-selection or Q-guidance leaves the underlying policy unchanged [15, 16]. Distilling critic-improved actions as terminal supervised labels disregards the flow structure of the velocity field [17]. The underlying obstacle is a mismatch: the critic evaluates executable clean action chunks, whereas the flow policy evolves through noisy intermediate states. Value improvement must therefore be expressed as a denoising-time velocity correction rather than a terminal action label.

To overcome these challenges, we propose *Q-Guided Value-Gradient Matching* (Q-VGM), an off-policy fine-tuning method for flow-matching VLA policies. Q-VGM resolves the above mismatch by casting policy improvement as stochastic optimal control over the $\pi_{0.5}$ denoising process, under which the optimal velocity correction is proportional to the gradient of a denoising-time value function. In practice, Q-VGM trains an action-sensitive stepwise critic on compact latent states from the frozen VLA prefix. It then projects each intermediate denoising state to a clean-action estimate via a one-step Euler look-forward, improves this estimate through iterative Q-gradient ascent with keep-best selection, and converts the induced displacement into a residual velocity target that directly supervises the velocity field. At inference time, the policy samples with the fine-tuned velocity field alone, fully amortizing critic guidance into the action expert.

Our contributions are as follows.

(1) A practical critic design for long-horizon tasks with sparse rewards. We develop an action-sensitive stepwise IQL critic for flow-matching VLA policies. By using compact RL-token states and stepwise chunk value learning, the critic provides reliable action-space value gradients $\nabla_A Q(s, A)$ under sparse rewards in long-horizon tasks.

(2) A value-gradient matching algorithm for policy extraction. Derived from an optimal-control view of the denoising process, Q-VGM converts critic-guided clean-action improvement into direct supervision of the flow velocity field. It combines look-forward clean-action estimation, iterative Q-gradient ascent with keep-best selection, and residual velocity matching, and fine-tunes a $\pi_{0.5}$ VLA entirely from off-policy rollout data.

(3) A real-robot iterative learn-from-experience pipeline. We validate Q-VGM in a train–deploy–collect–retrain loop. Starting from few-shot SFT, the policy improves through fully offline updates using only its self-generated rollouts, and each improved policy is redeployed to collect new experience. We evaluate this loop on four real-robot manipulation tasks, ranging from pick-and-place to fine-grained plug insertion, alongside LIBERO.

2 Related Work

Q-learning for diffusion and flow policies. Diffusion-QL [14] adds a Q-maximization objective to the diffusion policy loss and backpropagates through the denoising trajectory. Subsequent work improves stability and efficiency [18, 19], and QSM [20] relates the diffusion score to the action-gradient of a learned Q-function. These methods still rely on full-chain critic gradients, which are difficult to scale to billion-parameter flow-matching VLAs.

Q-guided action selection and distillation. Test-time selection methods sample multiple action candidates and execute the best one under a value estimate, learned verifier, or confidence score [15, 21, 22], improving inference behavior without updating the policy. Concurrent work QGF [23] steers sampling with look-forward Q-gradients at test time, likewise leaving the policy unchanged. PA-RL [17] goes further by performing local $\nabla_a Q$-based optimization and distilling the improved actions back into the policy, but this treats the critic-improved chunk as a terminal supervised label and does not supervise the velocity field.

Optimal-control and adjoint views. Reward fine-tuning of flow models can be formulated as KL-regularized stochastic optimal control [24, 25]. Adjoint Matching [26] derives regression targets through an adjoint formulation, and value-gradient guidance matches the optimal velocity correction to the gradient of a value function [26, 27]. Closest to our setting, Q-learning with Adjoint Matching [28] avoids full-chain Q backpropagation for diffusion and flow policies. We instead construct velocity corrections from iterative Q-gradient ascent with keep-best selection and train the VLA action expert by residual velocity matching.

3 Preliminaries

3.1 Flow Matching for VLA Models

We use $\pi_{0.5}$, a flow-based vision-language-action model [4, 5], as the base policy. Given a VLA conditioning context c, the action expert generates an action chunk

$$A = [a_0, a_1, \ldots, a_{H-1}].$$

We use $\tau = 0$ for Gaussian noise and $\tau = 1$ for the clean action chunk. Given noise $\epsilon \sim \mathcal{N}(0, I)$, the linear flow path is

$$x_\tau = (1-\tau)\epsilon + \tau A, \qquad \tau \in [0,1].$$

The action expert predicts a velocity field $v_\theta(x_\tau, \tau, c)$ trained by conditional flow matching.

At inference time, sampling integrates this velocity from noise to action over $0 = \tau_0 < \tau_1 < \cdots < \tau_K = 1$, starting from $x^{[0]} \sim \mathcal{N}(0, I)$ and ending at $x^{[K]} \approx A$. We use bracketed superscripts for discrete Euler steps.

3.2 KL-Regularized Policy Improvement

Given a reward signal $r(x_1)$ on clean actions and a reference policy p_{base}, the KL-regularized policy improvement objective [29] yields the *tilted distribution*:

$$p^\star(x_1) \;\propto\; p_{\mathrm{base}}(x_1) \cdot \exp\big(r(x_1)/\lambda\big), \tag{1}$$

which upweights high-reward actions while penalizing deviation from the reference, with λ controlling the KL penalty strength.

3.3 Value-Gradient Guidance for Flow Models

Following value-gradient guidance for flow alignment [27], policy improvement can be viewed as adding a residual velocity h to a base flow:

$$\dot{x}_\tau = v_{\mathrm{base}}(x_\tau, \tau) + h(x_\tau, \tau).$$

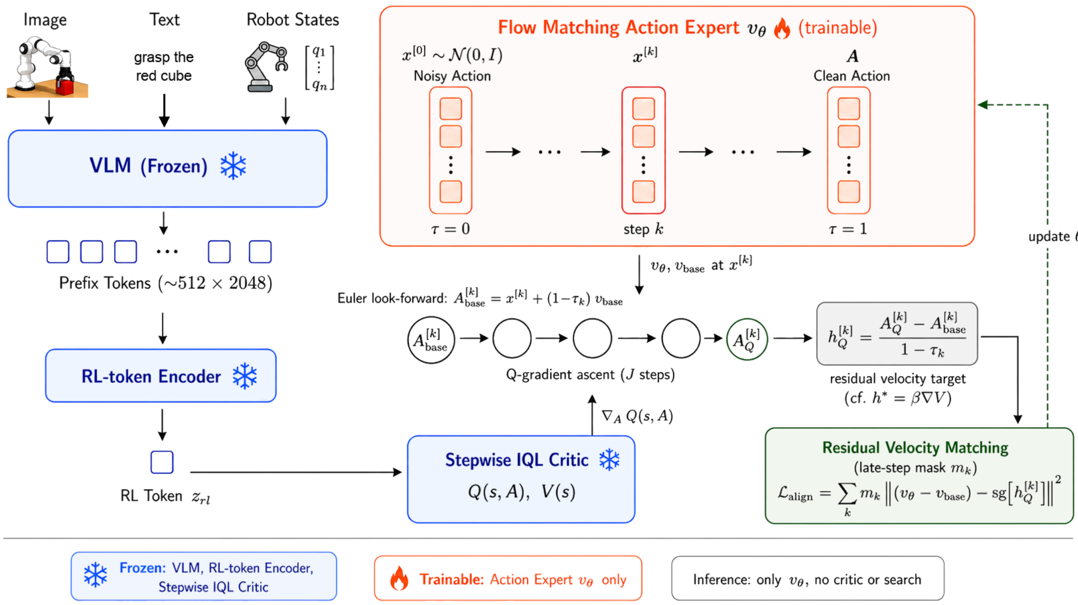

Figure 2: Overview of Q-Guided Value-Gradient Matching (Q-VGM). The frozen VLM prefix is compressed into an RL token that conditions a stepwise IQL critic (left). At each late denoising step, the Euler look-forward action estimate is improved by J steps of clipped Q-gradient ascent and converted into a residual velocity target $\hat{h}^{[k]}_Q$, which the action expert is trained to match (right). Only the action expert is trained; inference runs v_θ alone, without critic queries or search.

Here h is the velocity correction that changes the endpoint reached by the base denoising dynamics. Choosing h so that the corrected flow samples from the tilted distribution (1)—maximizing the terminal score $r(x_1)$ while penalizing deviation from the base process—is a stochastic optimal control problem [25–27]. Its solution is characterized by a denoising-time value function $V(x_\tau, \tau)$, the best regularized terminal score reachable from the intermediate state x_τ, and the optimal correction follows the value gradient:

$$h^\star(x_\tau, \tau) = \beta\, \nabla_x V(x_\tau, \tau), \qquad \beta = 1/\lambda. \tag{2}$$

At the clean endpoint, with terminal score $r(x_1)$,

$$V(x_1, 1) = r(x_1), \qquad \nabla_{x_1} V(x_1, 1) = \nabla_{x_1} r(x_1). \tag{3}$$

In Section 4, we instantiate this optimal-control framework for off-policy VLA fine-tuning, constructing the velocity correction directly from critic gradients with $r(x_1) = Q(s, x_1)$, where Q is a learned critic and s its state input.

4 Method

We fine-tune a pretrained flow-matching VLA with an off-policy critic. First, we train an action-sensitive off-policy critic on the rollout buffer (Section 4.1). Then, following the optimal-control view of Section 3.3, we construct value-gradient-guided velocity corrections from the critic and train the action expert's residual velocity to match them (Section 4.2).

4.1 Action-Sensitive Stepwise Chunk Critic

Value-gradient matching requires reliable action-space gradients $\nabla_A Q(s, A)$, where $A = [a_0, \ldots, a_{H-1}]$ is a continuous normalized clean-action chunk. We train such a critic off-policy on an offline buffer of policy rollouts, designing it to preserve the task semantics of the VLA prefix while remaining locally sensitive to the action chunk.

State representation. Following the $\pi_{0.5}$ architecture, we call the token sequence that the frozen VLM backbone produces from the visual observation, language instruction, and proprioceptive state the VLA *prefix*; it is the representation the flow-matching action expert conditions on. Feeding all prefix tokens into the critic is computationally impractical, so we compress the frozen prefix into an RL token $z_{\mathrm{rl}} \in \mathbb{R}^{2048}$ with an autoencoder, following Xu et al. [30], and form the critic state s by concatenating z_{rl} with a projection of the proprioceptive state (Section A). The autoencoder is pretrained by reconstructing the prefix; during critic training the backbone stays frozen while the encoder and critic heads are optimized jointly, keeping the reconstruction objective as a regularizer that grounds z_{rl} in the frozen prefix:

$$\mathcal{L}_{\mathrm{recon}} = \left\|\mathrm{decode}(z_{\mathrm{rl}}) - \mathrm{prefix}\right\|^2, \qquad \mathcal{L}_{\mathrm{critic}} = \mathcal{L}_{\mathrm{IQL}} + \alpha_{\mathrm{rec}}\mathcal{L}_{\mathrm{recon}}.$$

Because z_{rl} is high-dimensional relative to A, a naive critic may ignore the action input. We therefore re-inject A at every hidden layer of the critic, preserving the local action sensitivity that value-gradient matching relies on.

Stepwise IQL training. In long-horizon tasks with sparse rewards, a single value for the entire action chunk is too weak a supervision signal to fit an accurate Q-function. The critic therefore predicts stepwise values $\{Q^{(i)}(s, A)\}_{i=0}^{H-1}$, one per action position; when a scalar score is needed for value-gradient matching, we sum them into a chunk-level score, $Q(s, A) = \sum_{i=0}^{H-1} Q^{(i)}(s, A)$.

We train the critic with implicit Q-learning (IQL) [31]. An action-free value head predicts stepwise values $\{V^{(i)}(s)\}_{i=0}^{H-1}$. Each chunk-aligned transition in the buffer provides per-step rewards r_i, termination flags d_i, and the next-chunk critic state s'; with discount γ, each Q-head is trained toward a stepwise TD target that bootstraps to the next position inside the current chunk, and to the first position of the next chunk at the boundary:

$$y_i = \begin{cases} r_i + \gamma(1 - d_i)V^{(i+1)}(s), & 0 \le i < H - 1, \\ r_i + \gamma(1 - d_i)V^{(0)}(s'), & i = H - 1. \end{cases} \tag{4}$$

The value head is optimized with the IQL expectile loss and each Q-head with a TD loss toward y_i; full losses and masking details are in Section A. We use two Q-heads and take their minimum (clipped double Q-learning [32]), $Q^{(i)}(s, A) = \min\big(Q_1^{(i)}(s, A),\, Q_2^{(i)}(s, A)\big)$, for both target construction and value-gradient matching. Because the backup uses V rather than policy-sampled actions, critic training is fully off-policy.

4.2 Q-Guided Value-Gradient Matching

We instantiate the optimal-control framework (Section 3.3) with terminal reward $r(x_1) = Q(s, x_1)$ to fine-tune the flow-matching action expert. The optimal velocity correction $h^\star = \beta\,\nabla_x V$ (2) requires the value gradient at every denoising time, but the critic only provides action-space gradients $\nabla_A Q$ at clean actions. We therefore first derive a critic-based estimate of the value gradient on late denoising steps, then convert it into a velocity target for the action expert.

Estimating the value gradient with the critic. For each offline state s, we sample initial noise $x^{[0]} \sim \mathcal{N}(0, I)$ and roll out the current policy over K Euler steps with stop-gradient. At step k, the policy predicts $v_\theta^{[k]} = v_\theta(x^{[k]}, \tau_k, \cdot)$, and we advance $x^{[k+1]} = \mathrm{sg}[x^{[k]} + (\tau_{k+1} - \tau_k)\, v_\theta^{[k]}]$.

The correction $h^\star$ requires the value gradient $\nabla_x V(x^{[k]}, \tau_k)$ at the noisy state. On late denoising steps, little denoising remains, so the value of an intermediate state is essentially the critic score of the clean action it will reach, extending the boundary condition (3): $V(x^{[k]}, \tau_k) \approx Q(s, \hat{A}_{\mathrm{base}}^{[k]})$, where

$$\hat{A}_{\mathrm{base}}^{[k]} = x^{[k]} + (1 - \tau_k)\, v_{\mathrm{base}}(x^{[k]}, \tau_k, \cdot) \tag{5}$$

is the Euler look-forward endpoint under the frozen base velocity, which anchors the estimate to the behavior policy's distribution, within the critic's training support. Differentiating this composite gives

$$\nabla_x V(x^{[k]}, \tau_k) \;\approx\; \Big(\underbrace{I + (1 - \tau_k)\, \nabla_x v_{\mathrm{base}}}_{\partial \hat{A}_{\mathrm{base}}^{[k]} / \partial x^{[k]}}\Big)^\top \nabla_A Q(s, \hat{A}_{\mathrm{base}}^{[k]}).$$

On late steps, however, this Jacobian deviates from the identity only through the term $(1-\tau_k)\,\nabla_x v_{\text{base}}$, which is of order $1-\tau_k$ provided the base velocity field has bounded input sensitivity, a mild smoothness condition on the trained network. Substituting the identity for the Jacobian, which additionally avoids backpropagating through v_{base}, yields our estimate

$$\nabla_x V(x^{[k]}, \tau_k) \;\approx\; \nabla_A Q(s, \hat{A}^{[k]}_{\text{base}}). \tag{6}$$

Both approximations are exact at the boundary $\tau_k = 1$, where (3) holds and the look-forward step vanishes, and degrade as τ_k decreases; we therefore apply the construction below only on the last M denoising steps.

From gradient ascent to a velocity target. With the estimate (6), the optimal correction $h^\star = \beta\,\nabla_x V$ could be realized by a single scaled Q-gradient, but this commits to a fixed magnitude β. Instead, starting from $\hat{A}^{[k],0} = \hat{A}^{[k]}_{\text{base}}$, we perform J Q-gradient ascent steps:

$$\hat{A}^{[k],j+1} = \hat{A}^{[k],j} + \alpha\,\mathrm{clip}_G\Big(\nabla_A Q(s, \hat{A}^{[k],j})\Big), \qquad j = 0, \ldots, J-1, \tag{7}$$

where clip_G bounds the gradient magnitude. We retain the highest-valued candidate via keep-best selection:

$$j^\star = \arg\max_{j \in \{0,\ldots,J\}} Q(s, \hat{A}^{[k],j}), \qquad \hat{A}^{[k]}_Q = \hat{A}^{[k],j^\star}.$$

Since $j{=}0$ is the unmodified base prediction, keep-best falls back to the original action whenever ascent does not improve the value; it acts as a discrete line search on the local Q landscape, selecting a per-sample adaptive correction magnitude β_{eff}. The accumulated displacement, converted back into a velocity over the remaining time, gives the target

$$\hat{h}^{[k]}_Q = \frac{\hat{A}^{[k]}_Q - \hat{A}^{[k]}_{\text{base}}}{1-\tau_k}, \tag{8}$$

which shifts the flow from the base-policy destination toward the critic-improved action.

Residual velocity matching. To restrict training to these late steps, define the mask

$$m_k = \begin{cases} 1, & k \in \{K-M, \ldots, K-1\}, \\ 0, & \text{otherwise}, \end{cases}$$

with $M = 5$ in all experiments. We train the action expert so that its residual velocity matches $\hat{h}_Q$:

$$\mathcal{L}_{\text{align}} = \sum_{k=0}^{K-1} m_k \,\Big\| \big(v_\theta(x^{[k]}, \tau_k, \cdot) - v_{\text{base}}(x^{[k]}, \tau_k, \cdot)\big) - \hat{h}^{[k]}_Q \Big\|_2^2. \tag{9}$$

The denoising trajectory, base look-forward estimates, critic-improved candidates, and $\hat{h}^{[k]}_Q$ are treated as detached targets. Gradients flow only through the local prediction $v_\theta(x^{[k]}, \tau_k, \cdot)$. At inference, the policy samples with $v_\theta = v_{\text{base}} + h_\theta$, so the critic guidance is amortized into the action expert without test-time search or backpropagation through the denoising chain.

Algorithm 1 Q-Guided Value-Gradient Matching (Q-VGM)

Require: policy v_θ, frozen base v_{base}, frozen critic Q, late-step mask m_k, ascent steps J, step size α

for each training iteration **do**
  sample $s, x^{[0]} \sim \mathcal{N}(0, I)$; $\mathcal{L} \leftarrow 0$
  for $k = 0, \ldots, K-1$ **do**
    $v^{[k]} \leftarrow v_\theta(x^{[k]}, \tau_k, \cdot)$
    if $m_k = 1$ **then**
      $\hat{A}^{[k],0} \leftarrow x^{[k]} + (1-\tau_k) v_{\text{base}}(x^{[k]}, \tau_k, \cdot)$
      for $j = 0, \ldots, J-1$ **do**
        $g \leftarrow \mathrm{clip}_G(\nabla_A Q(s, \hat{A}^{[k],j}))$
        $\hat{A}^{[k],j+1} \leftarrow \hat{A}^{[k],j} + \alpha\, g$
      end for
      $j^\star \leftarrow \arg\max_{j \in \{0,\ldots,J\}} Q(s, \hat{A}^{[k],j})$
      $\hat{A}^{[k]}_Q \leftarrow \hat{A}^{[k],j^\star}$
      $\hat{h}^{[k]}_Q \leftarrow (\hat{A}^{[k]}_Q - \hat{A}^{[k],0})/(1-\tau_k)$
      $h^{[k]}_\theta \leftarrow v^{[k]} - v_{\text{base}}(x^{[k]}, \tau_k, \cdot)$
      $\mathcal{L} \mathrel{+}= \|h^{[k]}_\theta - \mathrm{sg}[\hat{h}^{[k]}_Q]\|_2^2$
    end if
    $x^{[k+1]} \leftarrow \mathrm{sg}[x^{[k]} + (\tau_{k+1} - \tau_k)\, v^{[k]}]$
  end for
  update θ using $\nabla_\theta \mathcal{L}$
end for

Table 1: Success rate (%) on LIBERO. Each suite is evaluated over 50 rollouts per task (500 episodes per suite).

Method	Spatial	Object	Goal	Long	Avg
Initial policy ($\pi_{0.5}$ few-shot SFT)	85.6	84.8	83.4	62.2	79.0
Test-time Q Selection	90.2	89.6	87.8	76.2	86.0
Test-time Q Guidance	93.8	90.6	89.8	80.4	88.7
Q-Improved Action Distillation	93.4	91.8	90.6	79.2	88.8
Diffusion-QL	79.8	74.6	80.4	55.6	72.6
Q-VGM (Ours)	**96.2**	**95.4**	**94.6**	**83.8**	**92.5**

5 Experiments

In this section, we evaluate Q-VGM by asking the following four research questions:

RQ1: How much can Q-VGM improve a few-shot SFT flow-matching VLA policy using only offline, self-generated rollouts?

RQ2: Does value-gradient matching provide a better critic-to-policy extraction mechanism than test-time guidance, test-time selection, action distillation, or direct Q-backpropagation?

RQ3: How sample-efficient is Q-VGM compared with online RL fine-tuning and offline alternatives?

RQ4: Does the iterative learn-from-experience loop transfer to real robots?

We answer these questions through simulation benchmarks, real-robot deployment, and ablation studies.

5.1 Experimental Setup

Across all settings, we initialize $\pi_{0.5}$ with few-shot SFT, assemble a fixed rollout dataset from evaluation rollouts of the SFT policy, train a stepwise IQL critic, and fine-tune the policy offline. Critic-based baselines share the same SFT checkpoint, rollout data, RL-token features, and critic; only critic use differs: *test-time Q selection*, which re-ranks sampled action chunks by value [15, 16]; *test-time Q guidance*, which applies inference-time $\nabla_A Q$ refinement to sampled actions, adapting the action-gradient operator of [20, 33] to the frozen shared critic; *Q-guided action distillation*, which amortizes critic-improved actions into the policy [17]; and *Diffusion-QL*, which backpropagates the critic through the denoising chain [14].

5.2 Simulation Evaluation

This section addresses RQ1–RQ3 on LIBERO [34]. LIBERO uses the four standard suites: Spatial, Object, Goal, and Long, with each task evaluated over 50 independent rollouts (500 episodes per suite). For critic training, we simply reuse the episodes logged during this standard evaluation of the SFT policy, together with the suite's benchmark-provided expert demonstrations.

Simulation results. Table 1 shows that Q-VGM raises average success from 79.0% to 92.5% on LIBERO, outperforming the test-time critic methods and distillation baselines on every suite. Diffusion-QL instead degrades the SFT policy, as backpropagating the critic gradient through the full denoising chain is unstable.

Sample efficiency (RQ3). Table 2 compares the data used for policy improvement on LIBERO-Spatial. Q-VGM obtains its improvement from roughly $400\times$ fewer rollout episodes than online RL fine-tuning, using only episodes already logged during the standard evaluation of the SFT policy and a sparse success reward.

Table 2: Sample efficiency on LIBERO-Spatial: data used for policy improvement. Success rates as reported by each paper, with different backbones and SFT initializations.

Method	Backbone	Type	RL data (LIBERO-Spatial)	Spatial SR
π_{RL} [8]	$\pi_{0.5}$	online PPO	$\approx$204,800 on-policy episodes	99.6
SimpleVLA-RL [35]	OpenVLA-OFT	online GRPO	$\approx$358,400 on-policy episodes	99.1
ARFM [36]	flow VLA	offline	benchmark demos, hand-designed dense reward	95.8
Q-VGM (Ours)	$\pi_{0.5}$	offline	500 evaluation episodes + 432 demos	96.2

Table 3: Real-robot success rates of the Q-VGM fine-tuned policy

Task	Success rate
Grasp Blue Cup	20/20
Handover Microphone	15/20
Move Pot	19/20

5.3 Real-Robot Iterative Learn-from-Experience

Sample-efficient real-world improvement. This section addresses RQ4 by evaluating whether Q-VGM can improve success rates through repeated collect–train–deploy rounds. We evaluate four tasks on a bimanual robot platform: three tabletop manipulation tasks and a fine-grained plug-insertion task that requires millimeter-level alignment between the plug and the socket. Table 3 reports the success rate of the Q-VGM fine-tuned policy over 20 trials per task on in-distribution initial configurations.

Figure 1 (right) contrasts the SFT policy and Q-VGM on the plug-insertion task from the same initial configuration: the SFT policy leaves a millimeter-level misalignment and fails, while the fine-tuned policy applies the learned velocity correction and completes the insertion.

5.4 Ablation and Analysis

We ablate critic-side and policy-side components of Q-VGM on LIBERO.

Critic-side ablations. Replacing the RL-token state with a ResNet image encoder causes the largest critic-side drop, from 92.5% to 87.4%, showing that value-gradient matching benefits from a VLA-grounded state representation. Per-layer action injection and the double-Q minimum also improve local action sensitivity and candidate selection.

Policy-side ablations. The policy-side variants show that the conversion from critic gradients to velocity targets must be stabilized. Keep-best avoids using a final gradient-ascent iterate when it overshoots, the late-step mask focuses guidance where look-forward estimates are closest to the critic's training distribution, and the frozen-base anchor keeps velocity targets tied to the behavior-policy support.

6 Limitations

Our method relies on a learned critic $Q(s, A)$ to provide action-space value gradients for value-gradient matching. This makes the reliability of $\nabla_A Q(s, A)$ a central limitation. The gradient signal is most trustworthy near actions supported by the offline rollouts; outside this region, critic errors can produce misleading velocity corrections. We mitigate this with gradient clipping and the keep-best mechanism that falls back to the base action when gradient ascent does not improve the value. A stronger safeguard would adaptively bound the path-space deviation from the pretrained flow policy by internalizing the trust region into the sampling dynamics [37].

A broader limitation is the scalability and generalization of the critic. As task horizons and environment diversity grow, Q-learning becomes increasingly difficult to scale [38]. A promising future direction is to combine world models with value estimation. A learned dynamics model can shorten

Table 4: Ablation study on LIBERO. We report average success rate (%) over 40 tasks.

Variant	Avg SR (%)
Full method	**92.5**
Critic-side variants	
ResNet encoder instead of RL token	87.4
No per-layer action injection	88.2
Single critic head	90.1
Policy-side variants	
No keep-best selection	88.6
All-step velocity alignment (m_k=1)	86.2
No frozen-base anchor	86.8

the effective TD horizon via multi-step rollouts, while the value function continues to provide gradient signals for value-gradient matching.

7 Conclusion

We propose an off-policy value-guided fine-tuning method for flow-matching VLA policies. Inspired by the optimal-control view that value gradients induce local velocity corrections, our method uses an action critic to improve clean-action estimates via iterative Q-gradient ascent with adaptive keep-best selection, and converts the resulting action improvement into a velocity correction for residual velocity matching. The update preserves the native velocity-field parameterization of flow matching. Experiments on LIBERO and real-robot tasks show consistent improvements over all same-backbone, same-critic baselines.

References

[1] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2023.

[2] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[3] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024.

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[5] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[6] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Matt Le, and Maximilian Nickel. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

[7] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.

[8] Kang Chen, Zhihao Liu, Tonghe Zhang, Zhen Guo, Si Xu, Hao Lin, Hongzhi Zang, Xiang Li, Quanlu Zhang, Zhaofei Yu, Guoliang Fan, Tiejun Huang, Yu Wang, and Chao Yu.

$\pi_{\texttt{RL}}$: Online RL fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025.

[9] Allen Z. Ren, Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy policy optimization. In *International Conference on Learning Representations*, 2025.

[10] Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. ReinFlow: Fine-tuning flow matching policy with online reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025.

[11] Mingyang Lyu, Yinqian Sun, Erliang Lin, Huangrui Li, Ruolin Chen, Feifei Zhao, and Yi Zeng. Reinforcement fine-tuning of flow-matching policies for vision-language-action models. *arXiv preprint arXiv:2510.09976*, 2025.

[12] Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-GRPO: Training flow matching models via online RL. *arXiv preprint arXiv:2505.05470*, 2025.

[13] Jeongjae Lee, Jinho Chang, Jeongsol Kim, and Jong Chul Ye. Reward score matching: Unifying reward-based fine-tuning for flow and diffusion models. *arXiv preprint arXiv:2604.17415*, 2026.

[14] Zhendong Wang, Jonathan J Hunt, and Mingyuan Zhou. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations*, 2023.

[15] Haoming Song, Delin Qu, Yuanqi Yao, Qizhi Chen, Qi Lv, Yiwen Tang, Modi Shi, Guanghui Ren, Maoqing Yao, Bin Zhao, Dong Wang, and Xuelong Li. Hume: Introducing System-2 thinking in visual-language-action model. *arXiv preprint arXiv:2505.21432*, 2025.

[16] Mitsuhiko Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. In *Conference on Robot Learning*, 2024.

[17] Max Sobol Mark, Tian Gao, Georgia Gabriela Sampaio, Mohan Kumar Srirama, Archit Sharma, Chelsea Finn, and Aviral Kumar. Policy agnostic RL: Offline RL and online RL fine-tuning of any class and backbone. *arXiv preprint arXiv:2412.06685*, 2024.

[18] Bingyi Kang, Xiao Ma, Chao Du, Tianyu Pang, and Shuicheng Yan. Efficient diffusion policies for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

[19] Tianyu Chen, Zhendong Wang, and Mingyuan Zhou. Diffusion policies creating a trust region for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, 2024.

[20] Michael Psenka, Alejandro Escontrela, Pieter Abbeel, and Yi Ma. Learning a diffusion model policy from rewards via Q-score matching. In *International Conference on Machine Learning*, 2024.

[21] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. In *Conference on Robot Learning*, 2025.

[22] Suhyeok Jang, Dongyoung Kim, Changyeon Kim, Youngsuk Kim, and Jinwoo Shin. Verifier-free test-time sampling for vision language action models. In *International Conference on Learning Representations*, 2026.

[23] Zhiyuan Zhou, Andy Peng, Charles Xu, Qiyang Li, Tobias Springenberg, Kevin Frans, and Sergey Levine. Test-time gradient guidance of flow policies in reinforcement learning. *arXiv preprint arXiv:2606.11087*, 2026.

[24] Masatoshi Uehara, Yulai Zhao, Kevin Black, Ehsan Hajiramezanali, Gabriele Scalia, Nathaniel Lee Diamant, Alex M Tseng, Tommaso Biancalani, and Sergey Levine. Fine-tuning of continuous-time diffusion models as entropy-regularized control. *arXiv preprint arXiv:2402.15194*, 2024.

[25] Wenpin Tang and Fuzhong Zhou. Fine-tuning of diffusion models via stochastic control: entropy regularization and beyond. *arXiv preprint arXiv:2403.06279*, 2024.

[26] Carles Domingo-Enrich, Michal Drozdzal, Brian Karrer, and Ricky T. Q. Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. In *International Conference on Learning Representations*, 2025.

[27] Zhen Liu, Tim Z. Xiao, Carles Domingo-Enrich, Weiyang Liu, and Dinghuai Zhang. Value gradient guidance for flow matching alignment. In *Advances in Neural Information Processing Systems*, 2025.

[28] Qiyang Li and Sergey Levine. Q-learning with adjoint matching. In *International Conference on Learning Representations*, 2026.

[29] Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.

[30] Charles Xu, Jost Tobias Springenberg, Michael Equi, Ali Amin, Adnan Esmail, Sergey Levine, and Liyiming Ke. RL token: Bootstrapping online RL with vision-language-action models. *arXiv preprint arXiv:2604.23073*, 2026.

[31] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations*, 2022.

[32] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*, 2018.

[33] Long Yang, Zhixiong Huang, Fenghao Lei, Yucun Zhong, Yiming Yang, Cong Fang, Shiting Wen, Binbin Zhou, and Zhouchen Lin. Policy representation via diffusion probability model for reinforcement learning. *arXiv preprint arXiv:2305.13122*, 2023.

[34] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, 2023.

[35] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. SimpleVLA-RL: Scaling VLA training via reinforcement learning. In *International Conference on Learning Representations*, 2026.

[36] Hongyin Zhang, Shiyuan Zhang, Junxi Jin, Qixin Zeng, Yifan Qiao, Hongchao Lu, and Donglin Wang. Balancing signal and variance: Adaptive offline RL post-training for VLA flow models. *arXiv preprint arXiv:2509.04063*, 2025.

[37] Yonghoon Dong, Kyungmin Lee, Changyeon Kim, Jaehyuk Kim, and Jinwoo Shin. Trust region Q adjoint matching. *arXiv preprint arXiv:2605.27079*, 2026.

[38] Seohong Park, Kevin Frans, Deepinder Mann, Benjamin Eysenbach, Aviral Kumar, and Sergey Levine. Horizon reduction makes RL scalable. In *Advances in Neural Information Processing Systems*, 2025.

[39] Mitsuhiko Nakamoto, Yuexiang Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-QL: Calibrated offline RL pre-training for efficient online fine-tuning. In *Advances in Neural Information Processing Systems*, 2023.

A Critic Architecture and Training Details

Critic architecture. The critic state is $s = \mathrm{LayerNorm}([z_{\mathrm{rl}} \parallel W_p\, p]) \in \mathbb{R}^{2304}$, where $z_{\mathrm{rl}} \in \mathbb{R}^{2048}$ is the cached RL token and W_p projects the proprioceptive state. Each Q-head uses per-layer action injection: the action chunk $A \in \mathbb{R}^{H \times d_a}$ is flattened and re-concatenated at every hidden layer (2 layers, dims $1024 \to 512$) so that action information is not washed out by the larger state dimensions. In the 7-DoF settings with $H{=}5$, this is a 35-dimensional flattened action input. A separate expectile value head of the same width predicts the stepwise values $V(s)^{(i)}$. Total: $\sim$3.9M parameters per Q-head.

RL-token pretraining. The autoencoder that produces z_{rl} is a 2-layer transformer encoder–decoder (2048-dim tokens, 8 heads). We pretrain it by MSE reconstruction of the frozen VLA prefix embeddings, reaching >0.95 cosine similarity on held-out data, and use these weights to initialize the encoder for critic training.

Stepwise IQL training. The critic is trained on clean action chunks $A_t = [a_{t,0}, \ldots, a_{t,H-1}] \in \mathbb{R}^{H \times d_a}$. From each offline rollout we form stepwise transitions $(s_t, A_t, \{r_{t,i}\}, s_{t+H}, \{d_{t,i}\})$ over chunk-aligned windows. The value head predicts $\{V^{(i)}(s)\}_{i=0}^{H-1}$ and is trained by expectile regression to the detached target critic. With expectile $\tau_{\mathrm{IQL}}{=}0.8$,

$$\rho_{\tau_{\mathrm{IQL}}}(u) = \begin{cases} \tau_{\mathrm{IQL}} u^2, & u \geq 0, \\ (1-\tau_{\mathrm{IQL}}) u^2, & u < 0, \end{cases} \qquad \mathcal{L}_V = \mathbb{E}_i \left[\rho_{\tau_{\mathrm{IQL}}} \left(\bar{Q}^{(i)}(s, A) - V^{(i)}(s) \right) \right],$$

where $\bar{Q}$ is the minimum over the two detached target Q-heads. Each Q-head is fit by the stepwise temporal-difference target y_i defined in (4), with

$$\mathcal{L}_Q = \mathbb{E}_{i,n} \left[\left(Q_n^{(i)}(s, A) - y_i \right)^2 \right], \qquad \mathcal{L}_{\mathrm{IQL}} = \mathcal{L}_V + \mathcal{L}_Q.$$

The boundary term is masked out when the next chunk is partial, and a per-position validity mask keeps positions up to and including the first terminal within a chunk. The target network is updated by an exponential moving average. Unlike conservative offline backups such as Cal-QL [39], which bootstrap on a sampled or dataset next action and add a CQL penalty against out-of-distribution overestimation, IQL bootstraps on the in-support value $V(s')$ and never queries an off-sample action, so it needs no conservative penalty. We keep the autoencoder reconstruction loss $\mathcal{L}_{\mathrm{recon}}$ (weight α_{rec}) as a regularizer throughout, so z_{rl} stays grounded in the VLA prefix.